A classification of well-behaved graph clustering schemes

Vilhelm Agdur

May 1, 2025

Abstract

1 Introduction

The problem of inferring community structure from a graph is one that appears in many different guises in different research areas. It is useful for everything from assigning research papers to subfields of physics [CR10], assessing the development of nano-medicine by studying patent cocitation networks [BAB12], studying brain function via the community structure of the connectome [Bet23], understanding how different bird species spread various plant seeds [Cos+16], to studying hate speech on Twitter [Evk+21].

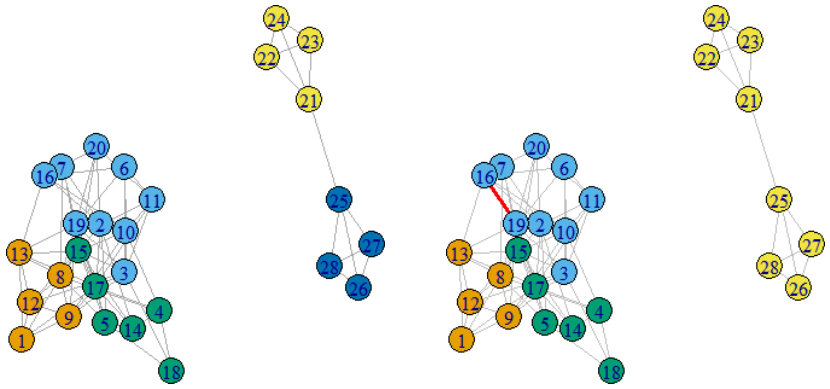
This problem is related to a broader problem of clustering, where the data may take the form of dissimilarity metric on items. In this guise, the study of clustering has a long history, with an early example being the book by Jardine and Sibson [JS71] on mathematical taxonomy from 1971. In fact, reading the introduction to that book already prefigures many of the issues discussed later in the literature, such as the difficulty of determining what exactly is meant by "community structure" or "clustering", and the lack of distinction between algorithms for finding partitions and the definitions of the structures we attempt to find.

Unfortunately, the development since then has seen the development of many directions of research and varying methods, without much contact between the different approaches. This has lead to a confusion of various methods for community detection on graphs, where there is rarely any clear theoretical justification for preferring one method over another.

In the literature on community detection on graphs, the most common approach is to define a quality function on partitions, and then treat community detection as an optimization problem.

One of the most popular methods in practice, namely modularity optimization [NG04], is of this type – it is the method used in all the examples given in the first paragraph. Other noteworthy methods in broadly the same strand of research are the flow-based infomap method motivated by information theory [RAB09], more statistically sound methods assuming a generative model [Pei14], novel graph neural network methods [Koj+23], but many more exist [Jav+18].

What the methods in this research direction have in common is that they are generally black boxes – they do not give you any human-interpretable way of answering the question of *why* two vertices were put in the same part. They are also generally hard to study in a rigorous mathematical way, resulting in few theoretical guarantees for their behaviour, and computationally, the optimization problem is normally not tractable.



(a) A graph G, with vertices coloured according to the modularity-maximising partition

(b) G with a single edge added, highlighted in red, and its new optimal partition

Figure 1: An illustration of the resolution limit issue with modularity.

For example, modularity suffers from a resolution limit, wherein it cannot "see" things happening in subgraphs that are small compared to the total graph, resulting in very unexpected behaviour [FB07]. In Figure 1 we see one illustration of this phenomenon – one would hope that what happens in one connected component cannot affect the other, but the resolution limit results in this undesirable behaviour.

In the literature on clustering based on dissimilarity measures, there has been much more attention to simpler methods, developing from the earliest studies of single-linkage clustering to more sophisticated versions of the same idea. In single-linkage clustering, we simply declare two objects to belong to the same part if they are sufficiently close, which leads to a very tractable and easily-analyzed method. However, it struggles with "chaining" effects, where two objects that are far apart are joined together by a chain of objects that are all close to each other.

This class of methods has received sophisticated theoretical analysis beginning with the work of Carlsson and Mémoli [CM13a], casting the problem of determining possible clustering methods in category-theoretical language as understanding properties of functors from the category of finite metric spaces to the category of sets with partitions. This line of research has also been extended into settings that cannot be represented as finite metric spaces, such as hypergraphs [**empty citation**] and directed graphs [**empty citation**].

These issues with the varying optimization based approaches to clustering lead us to investigating a more "axiomatic" approach to the problem – that is, we want to specify a few properties that the clustering method must have, and then see what methods there are that satisfy these properties. Other than of course giving us guarantees on the behaviour of our methods, this approach also gives us much more straightforward algorithms which run in polynomial time, since we are no longer doing optimization, but in some sense just applying a decision rule for when vertices should be in the same part or not.

Our particular approach to this is motivated by the results of Carlsson and Mémoli [CM13b] for the analogous problem of clustering finite metric spaces. Like in their setting, there are three properties that are of interest:

1. *Functoriality* is essentially a form of monotonicity under addition of edges or vertices

to the graph. Adding a new edge or new vertex should only ever make communities more closely connected, it should never split them.

2. *Excisiveness* is a very strict version of requiring that there be no resolution limit – instead we require that the method be idempotent on the parts it has found. That is, if we take some graph G, cluster it, and then look at the subgraph induced by any part, that graph has to be clustered into just a single large component, so we never discover new features at smaller scales.

3. *Representability* is a type of explainability of the clustering method. A representable clustering method $\Phi_{\mathfrak{R}}$ is determined by a set $\mathfrak{R}$ of simple graphs, and the rule that two vertices of a graph G will be in the same part if their neighbourhood looks like something in $\mathfrak{R}$. This also gives us a simple algorithm to actually compute the partitioning of a graph, that runs in polynomial time for each fixed $\mathfrak{R}$.

Our main result classifies exactly the relationship between these properties, and shows that all representable clustering schemes are tractable on a large class of sparse graphs:

Theorem 1.1 (Main results)**.** *A clustering scheme is representable if and only if it is excisive and functorial. Further, any such representable clustering scheme is computable in quadratic time on any graph class of bounded expansion.*

We also give a structural result for when two representable clustering schemes are equal, and use this to prove that there exist representable clustering schemes that are not finitely representable. Finally, we extend our definition of a representable clustering scheme into the setting of hierarchical clustering schemes. Here, instead of just assigning a single clustering to each graph, we give one clustering for each "scale", giving a spectrum of clusterings of the graph from coarsest to finest. In this setting, we find that the connection to excisiveness no longer works.

2 Preliminaries

The reader who is less familiar with category theory is invited to review the very few concepts from it that we use in Appendix A.

List of notation

1. We use capital Roman letters for graphs, e.g. G or H, and their vertex and edge sets are also capital Roman letters, e.g. V, W and E, F. Likewise, when considering a partitioned set, both its underlying set and its set of parts are capital Roman letters, e.g. V and P.

2. We use lowercase Roman letters for vertices or edges of graphs, e.g. v or e, for parts of a partitioned set, e.g. p or q, and for functions between sets and for morphisms, e.g. f or g. One exception to this is that we use lowercase Greek letters for morphisms from a graph R in a representing set $\mathfrak{R}$, e.g. ω.

3. For the function sending an edge to its vertex set, we use the character $\triangledown$, pronounced "e".

4. For functors, we use capital Greek letters, e.g. Φ, Π, Λ.

5. For clustering schemes of graphs, and also for representing sets for representable clustering schemes, we use Fraktur letters, e.g. $\mathfrak{C}$, $\mathfrak{R}$.

6. If $f : X \to Y$ is a set function, we write f_* for the function from 2^X to 2^Y that sends a subset of X to its image under f in Y.

3 Definitions of our terms

Definition 3.1. The category of hypergraphs $\mathcal{G}$ has as its objects hypergraphs, which we consider to be a triple $(V, E, \mathfrak{r})$ of a finite set V of vertices, a finite set E of edges that is disjoint from V, and a function $\mathfrak{r} : E \to 2^V$ assigning each edge its vertex set. Notice that this allows us to have multiple edges with the same edge set.

A morphism from $H = (V, E, \mathfrak{r})$ to $G = (V', E', \mathfrak{r}')$ is an injective set function $f : V \to V'$ such that, for every $e \in E$, there is an $e' \in E'$ such that $f_*(\mathfrak{r}(e)) = \mathfrak{r}'(e')$. That is, a morphism from H to G is a way of realising H as a sub-hypergraph of G.

Definition 3.2. The category of simple graphs $\mathcal{G}_s$ is the full subcategory of $\mathcal{G}$ whose objects are the simple graphs, that is, the graphs all of whose edges contain two vertices, and where no two edges have the same vertex set. That is, there are no loops or parallel edges.

Whenever we use just the word "graph", we mean a hypergraph, since those are more common for us.

Definition 3.3. For a graph $G = (V, E, \mathfrak{r})$ and a subset of its vertices $p \subseteq V$, we define the *restriction* of G to p, $G|_p$, by that its vertex set is p, and its edges are precisely those edges of G all of whose vertices are contained in p. It is easy to see that the set inclusion function $i : p \hookrightarrow V$ will be a morphism from $G|_p$ to G.

Definition 3.4. The category $\mathcal{P}$ of sets with overlapping partitions has as its objects finite sets V together with a set of parts $P \subseteq 2^V$.

A morphism in $\mathcal{P}$ from (V, P) to (W, Q) is a set function $f : V \to W$ such that for every $p \in P$, there exists a $q \in Q$ such that $f_*(p) \subseteq q$.

It is worth remarking here that this is a very permissive definition of a partitioned set. It does not require that the union of the parts cover the entire set, nor does it require that the parts be disjoint. In fact, it even permits the cases where $\emptyset \in P$ or $\emptyset = P$.

Remark 3.5. Notice that this definition does not exclude the case of one part properly containing another part. However, if we have a partitioned set (V, P) with two parts $p \subsetneq q$, we can always remove the part p to get a new partitioned set $(V, P \setminus \{p\})$, which will be isomorphic to (V, P) in $\mathcal{P}$ via the identity function on the set V. We will call such parts which are properly contained in other parts *spurious* parts.

So, up to isomorphism, we need not care about such parts – however, allowing them in our definition makes some of our constructions clearer, and so we choose to keep them in our definition. One might think that we should instead have picked a stronger notion of morphism, to make these be non-isomorphic, but as we will see in Example 4.10, this would break the functoriality of our generalized notion of connected components.

In particular, partitioned sets of the form $(V, \{V, W\})$ arise as the image of graphs under this connected components functor, and morphisms between graphs turn into morphisms from $(V, \{V, W\})$ to $(V, \{V, W'\})$ for arbitrary choices of W and W'. So the fact that the spurious part W was allowed to move around arbitrarily is in fact a feature, not a bug, of our definition.

Definition 3.6. For a partitioned set $\mathcal{A} = (V, P)$ and a subset p of its underlying set, we define the restriction of $\mathcal{A}$ to p, $\mathcal{A}|_p$, by that its underlying set is p, and its set of parts is given by

$$\{q \cap p \mid q \in P\},$$

that is, we take the intersection of each part of $\mathcal{A}$ with p. It is easy to see that the set inclusion function $i : p \hookrightarrow V$ is a morphism from $\mathcal{A}|_p$ to $\mathcal{A}$. Do we ever actually use this notion anywhere?

Definition 3.7. The category $\mathcal{P}_{\text{no}}$ of sets with non-overlapping partitions is the full subcategory of $\mathcal{P}$ whose objects have disjoint set covers. We may equivalently think of it as the category of sets equipped with partial equivalence relations, where the equivalence classes are the parts of the partitioned set.

The partial-ness of the equivalence relations here comes from the fact that we do not require every vertex to be contained in a part. This is just an artifact of our definitions, which lead to us saying that an isolated vertex belongs to no part, instead of belonging to a part of its own. While this is somewhat unnatural, the only "fix" to this would be to require a loop at every vertex of a graph, which comes with its own awkwardness.

Definition 3.8. A *(general) clustering scheme* is a function $\mathfrak{C}$ that sends hypergraphs to partitioned sets, with the same underlying set – that is, if $\mathfrak{C}$ sends $G = (V, E, \mathfrak{r})$ to $P = (W, P)$, we require that $V = W$. If the image of $\mathfrak{C}$ is in $\mathcal{P}$, we say that it is a *non-overlapping* clustering scheme.

If, in addition, $\mathfrak{C}$ becomes a functor from $\mathcal{G}$ to $\mathcal{P}$ by defining $\mathfrak{C}(f) = f$ as functions on the underlying set, we say that it is *functorial*.

Do we ever actually do this:? We may abuse our notation slightly and write $p \in \mathfrak{C}(G)$ when what we mean is that $\mathfrak{C}(G) = (V, P)$ and $p \in P$.

One good way to think of these definitions is as a kind of generalization of monotonicity. If we were restricting to only studying clustering of graphs on a fixed underlying set with all morphisms as the inclusion map, the statement that there is a morphism from G to H becomes precisely the statement that $G \subseteq H$, that is that G is less than H in the subgraph inclusion order. Likewise, if we restricted to only studying non-overlapping partitions, there is a natural partial order on the set of equivalence relations on the underlying vertex set, given by that $\sim < \approx$ if $\sim$ is a refinement of $\approx$, or equivalently if $\approx$ is a coarsening of $\sim$.

In this restricted setting, the statement that a clustering scheme is functorial becomes exactly the statement that it is monotone with respect to the inclusion order and the order on equivalence relations. By using the language of category theory to encode this information, we gain the ability to talk about things being "monotone under addition of vertices", not just under addition of edges, and it becomes easier to speak about monotonicity with respect to non-overlapping set partitions.

Definition 3.9. A clustering scheme $\mathfrak{C}$ is *excisive* if, for all graphs $G = (V, E, \mathfrak{r})$ and all parts $p \in \mathfrak{C}(G)$, p is a part of $\mathfrak{C}(G|_p)$.

So what this means is that if we "zoom in" on one part of a graph, the part we started with does not break apart into smaller pieces. Notice that this does not put any restrictions on what happens to any parts contained in or overlapping p.

In the setting of non-overlapping partitions, this property is more intuitive to state: If we cluster some graph, pick out one of the parts, and cluster just the subgraph induced by that part, we get back a trivial partition into a single part. Adapting this into the overlapping clustering world unfortunately gets us a less transparent definition.

4 Representability

Definition 4.1. Given a not necessarily finite set $\mathfrak{R}$ of graphs, we define the endofunctor $\Phi_{\mathfrak{R}}$ on $\mathcal{G}$ by that, for any graph $G = (V, E)$,

1. the vertices of $\Phi_{\mathfrak{R}}(G)$ are simply V,
2. the edges of $\Phi_{\mathfrak{R}}(G)$ are given by

$$E(\Phi_{\mathfrak{R}}(G)) = \bigcup_{R \in \mathfrak{R}} \hom(R, G),$$

   the set of all morphisms from a graph in $\mathfrak{R}$ into G,
3. and the edge-assigning function $\mathfrak{q}_{\Phi_{\mathfrak{R}}(G)} : E(\Phi_{\mathfrak{R}}(G)) \to 2^V$ simply assigns each morphism to its image, i.e. for $\omega \in \hom(R, G)$, we set $\mathfrak{q}_{\Phi_{\mathfrak{R}}(G)}(\omega) = \omega_*(R)$.

On morphisms, $\Phi_{\mathfrak{R}}$ is always just the identity on set functions, so if $f : G \to H$ is a morphism, $\Phi_{\mathfrak{R}}(f) = f$ as a set function.

In other words, more concretely, we get a hyperedge in $\Phi_{\mathfrak{R}}(G)$ for every copy of a graph in $\mathfrak{R}$ inside of G.

That this definition does in fact always give us a functor is something that needs to be checked – it is not immediate from the definition, even though we said in the definition that it is an endofunctor.

Lemma 4.2. *For any representing set $\mathfrak{R}$ of graphs, $\Phi_{\mathfrak{R}}$ is indeed an endofunctor on $\mathcal{G}$.*

Proof. That it always gives a graph and that it respects identity morphisms and composition of morphisms is easy to see. The only thing we actually need to check is that if $f : G \to G'$ is a morphism, then $\Phi_{\mathfrak{R}}(f) = f : \Phi_{\mathfrak{R}}(G) \to \Phi_{\mathfrak{R}}(G')$ is also a morphism.

Unwrapping this statement, what we need is that, for any two graphs $(G, E, \mathfrak{q})$ and $(G', E', \mathfrak{q}')$ and any morphism $f : G \to G'$, whenever e is an edge of $\Phi_{\mathfrak{R}}(G)$, there is an edge e' of $\Phi_{\mathfrak{R}}(G')$ such that $f_*(\mathfrak{q}_{\Phi_{\mathfrak{R}}(G)}(e)) = \mathfrak{q}'_{\Phi_{\mathfrak{R}}(G')}(e')$.

Now, this e is itself a morphism $\omega : R \to G$ for some $R \in \mathfrak{R}$, and so if we compose e with f we get a morphism $e' = f \circ e : R \to G'$, which by definition is an edge of $\Phi_{\mathfrak{R}}(G')$. That $f_*(\mathfrak{q}_{\Phi_{\mathfrak{R}}(G)}(e)) = \mathfrak{q}'_{\Phi_{\mathfrak{R}}(G')}(e')$ is then, if we unwrap our definitions, merely the statement that $f_*(e_*(R)) = (f \circ e)_*(R)$, which is trivial. □

What is going on in the proof of Lemma 4.2 is much clearer if we just think about it in terms of things being subgraphs of each other, forgetting the data of *how* exactly they are subgraphs. Then the statement that $\Phi_{\mathfrak{R}}$ is a functor boils down to the statement that if ω is a subgraph of H and H is a subgraph of G, then ω is a subgraph of G, and so $\Phi_{\mathfrak{R}}(G)$ must have all the edges $\Phi_{\mathfrak{R}}(H)$ has, since edges in $\Phi_{\mathfrak{R}}(G)$ represent the presence of a certain subgraph in G.

When constructing this theory for the case of non-overlapping partitions, the crucial step is to take the connected components of the graph $F_\Omega(G)$, and so get a functor taking G to a set partition. By generalizing the notion of connected components a bit, we get many more options that will output also overlapping partitions.

Definition 4.3. The functor $\Pi : \mathcal{G}_s \to \mathcal{P}_{\text{no}}$ which sends a simple graph to its partition into connected components is called the connected components functor.

Checking that this is indeed a functor is an easy exercise, and so we omit the proof – it essentially boils down to the statement that adding an edge to a graph cannot disconnect a connected component, or equivalently that connectedness is a monotone increasing property in the edges.

In order to generalize this notion to hypergraphs, let us notice that for simple graphs, taking the line graph of the graph and looking at its connected components changes nothing – the set of vertices incident to the edges of a connected component of the line graph is a connected component of the original graph.

For hypergraphs, we get more than one possible notion of line graph, and therefore we get more than one notion of connected components.

Definition 4.4. For each $k \in \mathbb{N} \cup \{\infty\}$, we define the k-line graph functor $\Lambda_k : \mathcal{G} \to \mathcal{G}_s$ as follows:

For each $G = (V, E, \mathfrak{v})$:

1. The vertices of $\Lambda_k(G)$ are the vertex sets of the edges of G. That is,

$$V(\Lambda_k(G)) = \{\mathfrak{v}(e) \mid e \in E\}.$$

   Notice that this means that if there are multiple edges in E with the same vertex set, we still only get one vertex of $\Lambda_k(G)$ corresponding to them.

2. there is an edge $uv \in E(\Lambda_k(G))$ whenever $|u \cap v| \geq k$, that is, if the two edges of G overlap in at least k vertices. So if $k = \infty$, we get a trivial case where the line graph has no edges.

We define, for a morphism $f : H \to G$ in $\mathcal{G}$, that $\Lambda_k(f) = f_* : 2^{V(H)} \to 2^{V(G)}$. This does at least send objects of the right type to objects of the right type, because the vertices of $\Lambda_k(H)$ and $\Lambda_k(G)$ are by construction sets of vertices of H and G respectively.

However, since $V(\Lambda_k(G))$ does not contain *every* subset of $V(G)$, we do need to check that for a vertex v of $\Lambda_k(H)$, $f_*(v)$ is indeed a vertex of $\Lambda_k(G)$. To see this, note that this v is by construction the vertex set of some edge $e \in E(H)$, and since f is a morphism, there exists some edge $e' \in E(G)$ such that $f_*(\mathfrak{v}_H(e)) = \mathfrak{v}_G(e')$ – and this $\mathfrak{v}_G(e')$ is a vertex of $\Lambda_k(H)$.

That this line graph functor is in fact a functor definitely requires a proof, since the statement is far from obvious.

Lemma 4.5. *For each k, Λ_k is in fact a functor from $\mathcal{G}$ to $\mathcal{G}_s$.*

Proof. That $\Lambda_k(G)$ is indeed a simple graph for each $G \in \mathcal{G}$ is clear, so suppose $f : G \to H$ is a morphism in $\mathcal{G}$. We need to check that $\Lambda_k(f) : \Lambda_k(G) \to \Lambda_k(H)$ is a morphism in $\mathcal{G}_s$.

So, to check this, there are two things we need to show: That $\Lambda_k(f)$ is injective as a set function, and that, for each each edge $uv \in E(\Lambda_k(G))$, there is an edge $f(u)f(v)$ in $E(\Lambda_k(H))$.

If we unwrap our definitions a bit, what we end up with is that this boils down to the following two statements, both of which are easily verified facts about the image of functions:

1. Whenever f is an injective function, f_* is as well.

2. If f is injective, then $|a \cap b| = |f_*(a) \cap f_*(b)|$.

That Λ_k sends identity morphisms to identity morphisms and respects composition of morphisms is easily seen from the corresponding statements about images of functions. $\square$

So, now we have the line graph functor, which gives us a simple graph which we can take connected components of. This nearly gets us where we want to be – except of course the connected components of the line graph of G will be a partitioning of the vertex sets of the edges of G, not a partitioning of the vertices of G. So our last step to getting a generalized connected components functor is to turn this into a partitioning of the vertices.

Definition 4.6. For any set X, let

$$\upsilon(X) = \bigcup_{x \in X} x,$$

that is, we send X to the union of all its elements. The reader may be momentarily confused that we are taking the union of elements of a set we did not construct as a set of sets – how do we know that the things we are taking a union of are actually sets, so this is well-defined? However, she can find solace in the fact that, under the axioms of ZFC, *all* objects are sets, and so this is well defined – and perhaps one of very few instances where "all objects are sets" is useful.

We define the part-union operation $\Upsilon : \mathcal{P} \to \mathcal{P}$ by that for each $\mathcal{A} = (V, P) \in \mathcal{P}$, we let $\Upsilon(\mathcal{A}) = \mathcal{B} = (\upsilon(V), \upsilon_*(P))$.

Unfortunately, this Υ is not in general a functor, since there is no way to define what it does on morphisms between partitioned sets. However, it will still be useful to us.

Example 4.7. Add example proving that Υ cannot be made into a functor.

With all this in hand, we are finally able to define the connected components of a hypergraph.

Definition 4.8. For each k, the k-ly connected components functor $\Pi_k : \mathcal{G} \to \mathcal{P}$ is given by that, for any $G = (V, E)$, the parts of $\Pi_k(G)$ are the parts of $(\Upsilon \circ \Pi \circ \Lambda_k)(G)$, and the underlying set of $\Pi_k(G)$ is V. On morphisms, $\Pi_k(f)$ is simply the same underlying set function – this is well defined, since clearly the underlying set of $\Pi_k(G)$ will be the same set as the vertex set of G.

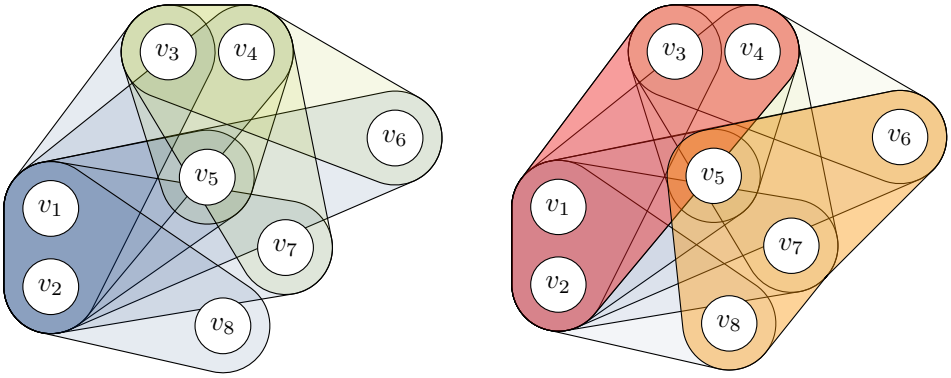
We say that a graph is *k-ly connected* if $\Pi_k(G)$ has the entire vertex set of G as a part, and call the parts of $\Pi_k(G)$ *k-ly connected components* of G.

The reason we do not simply say that $\Pi_k(G) = \Upsilon(\Pi(\Lambda_k(G)))$ is that if there is an isolated vertex of G, this vertex will not be in any edge of $\Lambda_k(G)$, and so when we apply $\Upsilon \circ \Pi$, it would not be in the underlying set. Thus our definition is the necessary choice to not forget about the isolated vertices entirely – we do still end up declaring that they are in no part, but there seems to be no nice way of defining that problem away.

That this one is a functor also requires a proof, since it is defined as a composition of some things one of which is not itself a functor. However, the fact that the first two things are functors will be helpful in the proof.

Lemma 4.9. *For each k, Π_k is a functor.*

Proof. As usual, once we have checked that Π_k does send morphisms to morphisms, it is trivial that it respects identity morphisms and composition.



(a) A graph G (b) A graph H

Figure 2: An illustration of two graphs in $\mathcal{G}$, with H a subgraph of G, where the image of this morphism under Λ_2 gives a surprising morphism in $\mathcal{P}$, as explained in Example 4.10. Note that H has all the edges of G, and an additional two edges, illustrated in red and orange – the other edges have had their colour faded to help clarity, but are still present.

So, suppose $G = (V, E)$ and $H = (W, F)$ are two graphs, and $f : V \to W$ is a morphism from G to H. Let $E' = \{\mathfrak{V}(e) \mid e \in E\}$ and F' similarly be the set of vertex sets of edges of H.

Now let $\mathcal{A} = (E', P) = \Pi(\Lambda_k(G))$, $\mathcal{B} = (F', P') = \Pi(\Lambda_k(G))$ be the connected components of the k-line graphs of G and H respectively. Further let $\tilde{\mathcal{A}} = (V, Q) = \Upsilon(\mathcal{A})$ and $\tilde{\mathcal{B}} = (W, Q') = \Upsilon(\mathcal{B})$.

What we need to show is that f is a morphism from $\Upsilon(\mathcal{A})$ to $\Upsilon(\mathcal{B})$ in $\mathcal{P}$. This concretely means that we need to show that, for each $q \in Q$, there is a $q' \in Q'$ such that $f_*(q) \subseteq q'$.

Now, for each $q \in Q$, there is a part $p \in P$ such that $q = \bigcup_{e \in p} e$. By functoriality of Λ_k and Π, we already know that f_* is a morphism from $\mathcal{A}$ to $\mathcal{B}$. This means that there has to exist a $p' \in P'$ such that $(f_*)_*(p) \subseteq p'$.

Now, by definition

$$(f_*)_*(p) = \{f_*(e) \mid e \in p\},$$

so if we let $q' = \bigcup_{e \in p'} e$, this must lie in Q' by construction, and we get that

$$f_*(q) = f_*\left(\bigcup_{e \in p} e\right) = \bigcup_{e \in p} f_*(e) = \bigcup_{e' \in (f_*)_*(p)} e' \subseteq \bigcup_{e' \in p'} e' = q',$$

as desired. □

Example 4.10. At this point, we can illustrate why our class of morphisms of partitioned sets has to be so large. Consider the graphs G and H shown in Figure 2. G has eight vertices, and an edge $v_1v_2v_i$ for each $i > 2$, illustrated in blue, and an edge $v_3v_4v_j$ for $j = 5, 6, 7$, illustrated in green. H is a supergraph of G, with all the edges of G and two additional edges, $v_1v_2v_3v_4$ and $v_5v_6v_7v_8$, illustrated in red and orange. So that the identity function on the vertex set is a morphism from G to H is clear.

Now consider the images of these two graphs under Π_2. In G, the blue edges will form a clique in the 2-line graph, and the green edges will form another clique. No green edge overlaps any blue edge in more than a single vertex, so these two cliques will be the connected components, and so the parts of $\Pi_2(G)$ will be the entire vertex set and $\{v_3, v_4, v_5, v_6, v_7\}$.

The addition of the red edge to H corresponds, in the line graph, to adding a new vertex with an edge to every vertex in the two cliques, so they merge into a new connected component, whose image under υ will be the entire vertex set. The orange edge, however, will not be connected to any of the vertices in the two cliques, and so will form a new connected component, whose image under υ will be $\{v_5, v_6, v_7, v_8\}$.

So we have seen that there must be a morphism from $(V, \{V, \{v_3, v_4, v_5, v_6, v_7\}\})$ to $(V, \{V, \{v_5, v_6, v_7, v_8\}\})$ in $\mathcal{P}$, which may at first have seemed like a bug in our definition, but is in fact a necessary feature.

Definition 4.11. A clustering scheme $\mathfrak{C}$ is *representable* if $\mathfrak{C} = \Pi_k \circ \Phi_{\mathfrak{R}}$ for some representable endofunctor $\Phi_{\mathfrak{R}}$ and some $k \geq 1$. Notice that this means that all representable clustering schemes are functorial, being the composition of two functors.

Given a set $\mathfrak{R}$ of graphs and an integer k, we write $\Pi_{\mathfrak{R},k}$ for the representable clustering scheme induced by $(\mathfrak{R}, k)$, that is, $\Pi_{\mathfrak{R},k} = \Pi_k \circ \Phi_{\mathfrak{R}}$.

Example 4.12. If we restrict to considering only simple graphs, it is very easy to see that the connected components functor Π is representable, since we can simply take $\mathfrak{R} = \{K_2\}$ and $k = 1$ – then $\Phi_{\mathfrak{R}}$ just sends a graph to itself, and of course connected components of the 1-line graph of a simple graph and connected components of the graph itself are the same thing.

For hypergraphs, we need to take $\mathfrak{R} = \{E_n \mid n \in \mathbb{N}\}$, where E_n is the hypergraph with n vertices and a single edge containing all the vertices. Then all $\Phi_{\mathfrak{R}}$ will do to a graph is to replace each edge with k vertices with $k!$ copies of itself, which does not change the set of vertex sets of edges, and so again we recover the usual notion of connected components by looking at the connected components of the 1-line graph of the graph.

In fact any set $\mathfrak{R}$ of connected graphs containing E_n for each n will represent Π_1 – but note that they will in general give inequivalent endofunctors on $\mathcal{G}$.

One somewhat surprising fact is that there are actually representable clustering schemes that are not finitely representable.

Theorem 4.13. *There exists a representable clustering scheme $\Pi_{\mathfrak{R},1}$ which is not* finitely *representable, that is, it is not equal to $\Pi_{\mathfrak{G},1}$ for any finite set of graphs $\mathfrak{G}$.*

A fortiori, there exists a representable endofunctor that is not finitely representable.

In order to give a proof of this, we first need a structural result about how these endofunctors behave. Let us say that an edge in a hypergraph which contains all vertices is a *spanning edge*, and a hypergraph which has a spanning edge is *spanned*.

Remark 4.14. For any $R \in \mathfrak{R}$, it is trivial to see that $\Phi_{\mathfrak{R}}(R)$ must be spanned – specifically we can take the identity morphism on R as the spanning edge.

In fact, which graphs are sent to spanned graphs by $\Phi_{\mathfrak{R}}$ essentially determines what it does as a functor. Similarly, if we are interested in the weaker equivalence of inducing the same clustering scheme, this is determined by which graphs are sent to a connected graph.

Lemma 4.15. *For any set $\mathfrak{R}$ of graphs and any graph $G = (V, E, \forall)$, it holds that $\Phi_{\mathfrak{R}\cup\{G\}} = \Phi_{\mathfrak{R}}$ if and only if G is sent to a spanned graph by $\Phi_{\mathfrak{R}}$.*

Proof. Suppose $\Phi_{\mathfrak{R}\cup\{G\}} = \Phi_{\mathfrak{R}}$. That $\Phi_{\mathfrak{R}\cup\{G\}}(G)$ is spanned is immediate by Remark 4.14, and so since $\Phi_{\mathfrak{R}\cup\{G\}} = \Phi_{\mathfrak{R}}$, $\Phi_{\mathfrak{R}}(G)$ is spanned, as desired.

Now suppose $\Phi_{\mathfrak{R}}(G)$ is spanned, and let H be some arbitrary simple graph. We wish to show that $\Phi_{\mathfrak{R}\cup\{G\}}(H) = \Phi_{\mathfrak{R}}(H)$.

That edges of $\Phi_{\mathfrak{R}}(H)$ must also be edges of $\Phi_{\mathfrak{R}\cup\{G\}}(H)$ is obvious, since a morphism from some graph in $\mathfrak{R}$ into H is also a morphism from a graph in $\mathfrak{R}\cup\{G\}$ into H, so all we need to show is that edges of $\Phi_{\mathfrak{R}\cup\{G\}}(H)$ are also edges of $\Phi_{\mathfrak{R}}(H)$. Thus, suppose e is an edge of $\Phi_{\mathfrak{R}\cup\{G\}}(H)$ – by construction, this edge is a morphism ω from some $R \in \mathfrak{R}\cup\{G\}$ into H, whose image is the vertex set of this edge.

If this R is not G, then we are done. If it is G, we recall that $\Phi_{\mathfrak{R}}(G)$ is spanned, and so we can take e' to be a spanning edge of $\Phi_{\mathfrak{R}}(G)$. Again, this edge is by construction a morphism ω' from some R' – this time an $R' \in \mathfrak{R}$ – into G, whose image, since it is spanning, must be the entirety of G.

So, if we compose $\omega' : R' \to G$ and $\omega : G \to H$, we get a morphism from R' into H, which by definition is an edge of $\Phi_{\mathfrak{R}}(H)$, and since ω' was onto, the image of this composition must be the same as the image of ω, and so this edge has the right vertex set, and we are done. □

Lemma 4.16. *For any set of graphs $\mathfrak{R}$ and any graph G, if $\Pi_{\mathfrak{R},k} = \Pi_{\mathfrak{R}\cup\{G\},k}$, then $\Phi_{\mathfrak{R}}(G)$ is k-ly connected. The reverse implication holds only if $k = 1$.*

Proof. Suppose $\Pi_{\mathfrak{R},k} = \Pi_{\mathfrak{R}\cup\{G\},k}$. We know from Remark 4.14 that $\Phi_{\mathfrak{R}\cup\{G\}}(G)$ is spanned, which immediately gives that it is k-ly connected. So since $\Pi_{\mathfrak{R},k} = \Pi_{\mathfrak{R}\cup\{G\},k}$, $\Phi_{\mathfrak{R}\cup\{G\}}(G)$ and $\Phi_{\mathfrak{R}}(G)$ have to have the same k-ly connected components, and therefore $\Phi_{\mathfrak{R}}(G)$ is also k-ly connected, as desired.

In the other direction, for the $k = 1$ case, suppose $\Phi_{\mathfrak{R}}(G)$ is 1-ly connected. This means $\Upsilon(\Pi(\Lambda_1(\Phi_{\mathfrak{R}}(G))))$ has a part containing every vertex, and by definition of Υ and Π, this means there is some connected component p of $\Lambda_1(\Phi_{\mathfrak{R}}(G))$ whose edges together contain every vertex. Let us call these edges $e_1, e_2, \ldots, e_n$, and observe that for each of these edges there exists an $R_i \in \mathfrak{R}$ and an $\omega_i : R_i \to G$, where $\mathrm{im}(\omega_i) = \mathfrak{V}(e_i)$.

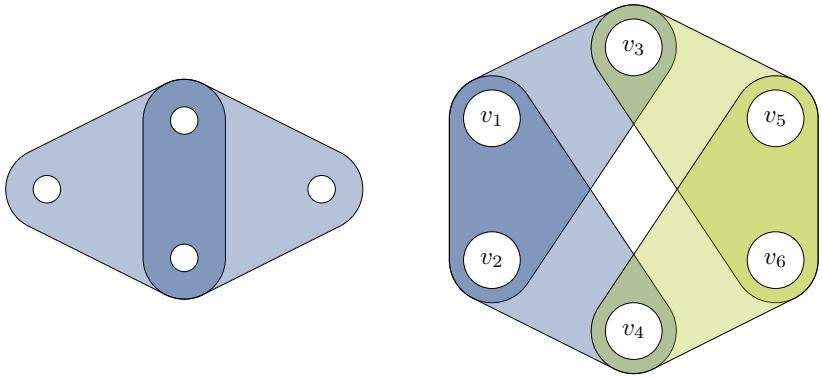
So let H be some arbitrary graph, for which we wish to show that $\Pi_{\mathfrak{R},1}(H) = \Pi_{\mathfrak{R}\cup\{G\},1}(H)$. It is easily seen that $\Phi_{\mathfrak{R}}(H)$ is a subgraph of $\Phi_{\mathfrak{R}\cup\{G\}}(H)$, so it will suffice to show that whenever there is a path in the line graph of $\Phi_{\mathfrak{R}\cup\{G\}}(H)$ between two edges e, e', there is a path between them also in $\Phi_{\mathfrak{R}}(H)$.

To lessen the potential confusion of terminology, we will from now on refer to the vertices of the line graph as "edges" and avoid using the word edge for the edges of the line graph, only referring to paths in the line graph and to edges (i.e. line-graph-vertices) being adjacent in such paths. The word "vertex", similarly, refers exclusively to a vertex of H.

So suppose we have such edges e, e' and a path between them in $\Lambda_1(\Phi_{\mathfrak{R}\cup\{G\}}(H))$. If this path does not contain any edge induced by a copy of G in H, then the same path exists also in $\Lambda_1(\Phi_{\mathfrak{R}}(H))$, and so we are done.

So, assume f is such an edge on this path, and say $\omega : G \to H$ is the morphism creating this edge. Let e_p be the edge previous to f on this path, and e_s be the edge subsequent to f. Since these edges are adjacent, there has to exist $v_p \in \mathfrak{V}(e_p) \cap \mathfrak{V}(f)$ and $v_s \in \mathfrak{V}(f) \cap \mathfrak{V}(e_s)$.

Now, since f arises as a copy of G in H, and the line graph of G has a connected component covering it, whose members are the edges e_i, the following has to be true: There exists $i_1, i_2, \ldots, i_m$ such that $v_p \in \omega(\mathfrak{V}(e_{i_1}))$, $v_s \in \omega(\mathfrak{V}(e_{i_m}))$, and $e_{i_1}, e_{i_2}, \ldots, e_{i_m}$ form a path in the line graph of G.



(a) The graph G (b) The graph H

Figure 3: Consider the situation where $\mathfrak{R}$ consists of just one graph on three vertices with one edge containing all three vertices, and G and H are as in the figures. It is clear that $\Phi_{\mathfrak{R}}(G)$ is k-ly connected. $\Lambda_2(\Phi_{\mathfrak{R}}(H))$ has two connected components, one containing the blue edges and one containing the green edges, and so $\Pi_{\mathfrak{R},2}(H)$ will have two parts, $\{v_1, v_2, v_3, v_4\}$ and $\{v_3, v_4, v_5, v_6\}$. However, when we add G into $\mathfrak{R}$, this will add two new edges into $\Phi_{\mathfrak{R}\cup\{G\}}(H)$, one containing v_1, v_2, v_3, v_4 and one containing v_3, v_4, v_5, v_6. These two edges overlap in v_3 and v_4, and so they form a connected component of $\Lambda_2(\Phi_{\mathfrak{R}\cup\{G\}}(H))$, which will be sent to a part containing all vertices. Thus $\Pi_{\mathfrak{R}\cup\{G\},2}(H)$ is not equal to $\Pi_{\mathfrak{R},2}(H)$.

So, if we compose the morphisms ω_i with ω, we get a collection of morphisms from graphs in $\mathfrak{R}$ into H – that is, edges in $\Phi_{\mathfrak{R}}(H)$. Call these edges $e'_1, e'_2, \ldots, e'_n$. Since $v_p \in \mathfrak{r}(e_p)$ and $v_p \in \mathfrak{r}(e'_{i_1})$, we must have that e_p is adjacent to e'_{i_1}, and similarly e_s is adjacent to e'_{i_m}. Notice that this is where we use the fact that $k = 1$ – this is the step that fails for $k > 1$.

It is easy to see that the rest of the edges on the path $e_{i_1}, e_{i_2}, \ldots, e_{i_m}$ are also still adjacent, and so we can replace f on the path from e to e' by this longer path, getting a path that does not use f, and so is a path also in $\Phi_{\mathfrak{R}}(H)$, which is what we needed to show.

That the reverse implication fails for $k > 1$ is shown by the example in Figure 3. □

With this result in hand, we can now give a proof of the existence of a representable but not finitely representable endofunctor.

Proof of Theorem 4.13. Let, for each i, R_i be the simple graph which consists of a triangle with a tail of i vertices, as shown in Figure 4, and let $\mathfrak{R} = \{R_i \mid i \in \mathbb{N}\}$. We claim that $\Pi_{\mathfrak{R},1}$ is not equal to $\Pi_{\mathfrak{G},1}$ for any finite $\mathfrak{G}$.

To prove this, let us first define

$$\bar{\mathfrak{R}} = \{G \in \mathcal{G} \mid \Phi_{\mathfrak{R}}(G) \text{ is 1-ly connected.}\}$$

as the set of all graphs which $\Phi_{\mathfrak{R}}$ sends to 1-ly connected graphs. By Lemma 4.16, $\Pi_{\bar{\mathfrak{R}},1} = \Pi_{\mathfrak{R},1}$, and $\bar{\mathfrak{R}}$ is the greatest set with this property.

It is not hard to see that any graph in $\bar{\mathfrak{R}}$ has to contain a triangle, since $\Phi_{\mathfrak{R}}$ will send any triangle-free graph to a graph with no edges, which is obviously not connected.

Now suppose for contradiction that $\mathfrak{G}$ is a finite set of graphs such that $\Phi_{\mathfrak{G}} = \Phi_{\bar{\mathfrak{R}}}$. Lemma 4.16 implies that we must have $\mathfrak{G} \subset \bar{\mathfrak{R}}$, since if there were a $G \in \mathfrak{G} \setminus \bar{\mathfrak{R}}$ it'd be sent

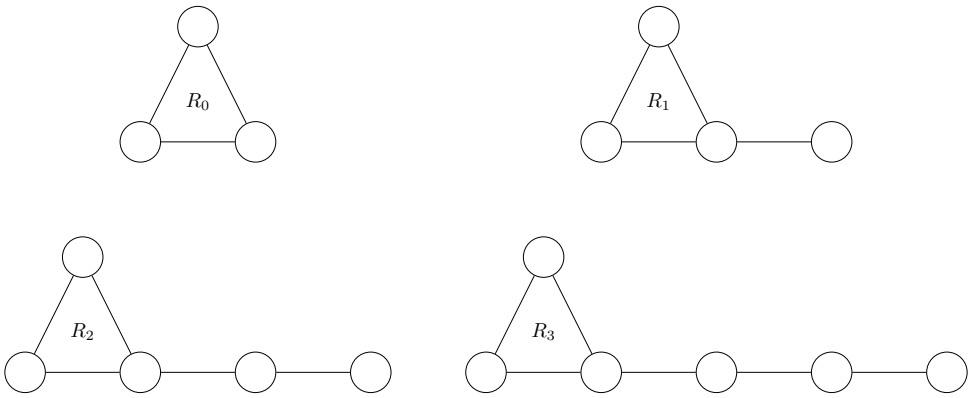


Figure 4: The family of graphs $\{R_i\}_{i=0}^{3}$.

to a 1-ly connected graph by $\Phi_{\mathfrak{G}}$, and thus also by $\Phi_{\bar{\mathfrak{R}}}$, since we've assumed $\Phi_{\mathfrak{G}} = \Phi_{\bar{\mathfrak{R}}}$ – but we chose $\bar{\mathfrak{R}}$ so that it already contains everything sent to a connected graph by $\Phi_{\bar{\mathfrak{R}}}$.

So, let

$$r = \max_{\substack{G \in \mathfrak{G} \\ G \text{ simple}}} \max_{u \in V(G)} \min_{\substack{v \in V(G) \\ v \text{ is in a triangle}}} d(u, v),$$

where $d(u, v)$ is the distance between u and v in G. That is, r is the furthest any vertex in any graph in $\mathfrak{G}$ is from a triangle. We claim that $\Phi_{\mathfrak{G}}$ will not send R_{r+1} to a 1-ly connected graph, proving it is not equal to $\Phi_{\bar{\mathfrak{R}}}$, giving our contradiction.

That this is so is actually easy to see – if R_{r+1} were sent to a 1-ly connected graph, that must in particular mean there is an edge in $\Phi_{\mathfrak{G}}(R_{r+1})$ containing the tail vertex and its neighbour. So there is some morphism from some $G \in \mathfrak{G}$ that hits both vertices – and clearly there is never a morphism from a non-simple graph into a simple graph, and so this G must be simple.

Now, we know that G contains a triangle, which has to be sent by this morphism onto the triangle in R_{r+1}, since that is the only place it can go – and since the morphism is by definition injective, this G cannot contain more than one triangle, since otherwise both triangles would have to be sent onto the one triangle in R_{r+1}. Then, however, this morphism cannot also hit the tail of R_{r+1} – the tail is further away from the triangle than any vertex in any simple $G \in \mathfrak{G}$ is from a triangle. Thus, no such morphism can exist, and we are done. □

Remark 4.17. The construction in our proof of course works equally as well replacing the triangle by any connected graph that is not a line. So we have found a large family of examples of classes of graphs that give representable but not finitely representable endofunctors.

This gives rise to a more general structural question. If we say that a graph class $\mathcal{C}$ is *representable* if there exists some $\mathfrak{R}$ such that $\bar{\mathfrak{R}} = \mathcal{C}$, we can ask ourselves which classes of graph are representable, and which are finitely representable. Our proof above essentially relied on showing that the class of connected triangle-free graphs is representable, but not finitely representable.

We have shown before that the connected simple graphs are representable, with $\mathfrak{R} = \{K_2\}$, and a similar construction will get the classes of connected hypergraphs of different

uniformities. The example we gave in the proof easily generalises to showing that the graphs of girth at most k are representable but not finitely representable, by extending from triangles with tails to all cycles of length at most k with tails.

The class of non-planar graphs is also representable, with itself as a representation, since there can of course be no morphism from a non-planar graph into a planar graph. Whether there exists a finite representing set is a potentially interesting question, which we leave unanswered.

5 Equivalence of representability and excisiveness

In the $k = 1$ case, where we get non-overlapping partitions, it turns out that representability is equivalent to excisiveness plus functoriality, and so we can give a complete characterization of representable clustering schemes in this case. This is known from add citation.

In the case where $k > 1$, interesting things start happening, and so we will need a weaker notion than equality of clustering schemes.

Definition 5.1. For any two clustering schemes $\mathfrak{C}_1, \mathfrak{C}_2 : \mathcal{G} \to \mathcal{P}$ we say that $\mathfrak{C}_1$ *refines* $\mathfrak{C}_2$ if the following two properties hold for every $G \in \mathcal{G}$:

1. Every part p of $\mathfrak{C}_1(G)$ is contained in some part p' of $\mathfrak{C}_2(G)$.
2. Every part p of $\mathfrak{C}_2(G)$ is also a part of $\mathfrak{C}_1(G)$.

Remark 5.2. Essentially, what this notion is saying is that for every graph G, $\mathfrak{C}_1(G)$ is $\mathfrak{C}_2(G)$, except possibly with some spurious parts added. Thus, this is a weaker notion than outright equality of clustering schemes.

However, this means that if $\mathfrak{C}_1$ refines $\mathfrak{C}_2$, it holds for every graph $G = (V, E)$ that the identity function on V is an isomorphism between $\mathfrak{C}_1(G)$ and $\mathfrak{C}_2(G)$. Thus, if $\mathfrak{C}_1$ refines $\mathfrak{C}_2$ and both are functorial, the two clustering schemes are in fact naturally isomorphic as functors from $\mathcal{G}$ to $\mathcal{P}$.

Remark 5.3. If both $\mathfrak{C}_1$ and $\mathfrak{C}_2$ send graphs to non-overlapping partitions, it is easily seen that $\mathfrak{C}_1$ refines $\mathfrak{C}_2$ if and only if $\mathfrak{C}_1 = \mathfrak{C}_2$, so it should not be too surprising that we get refinement of clustering schemes instead of equality when we generalize from the non-overlapping case.

Theorem 5.4. *Any representable clustering scheme is excisive and functorial. Any excisive and functorial clustering scheme is refined by a representable clustering scheme, and so in particular is isomorphic to a representable clustering scheme.*

We divide the proof of Theorem 5.4 into a collection of lemmas and examples, with the lemmas proving the stated implications and the examples demonstrating the existence of clustering schemes with every combination of properties that our lemmas do not forbid. We have already seen from Lemma 4.2 that representability implies functoriality, so it remains to see that it also implies excisiveness, and then to show that every excisive and functorial clustering scheme is refined by a representable clustering scheme.

A representable clustering scheme is by definition a composition of a representable endofunctor with the connected components functor – it turns out that in general, composing a representable endofunctor with any excisive clustering scheme will give another excisive clustering scheme, so this allows us to break the problem of showing that representable clustering schemes are excisive into two parts.

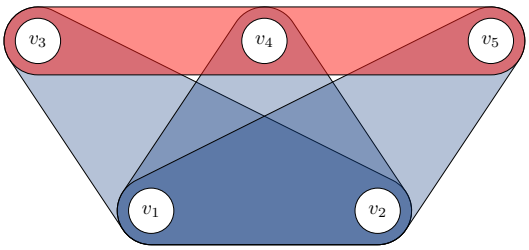


Figure 5: An example of a graph G with a connected component p (here, the entire vertex set) where p'' is a strict subset of p', in the notation of the proof of Lemma 5.7. In particular, we note that p'' consists of the three blue edges, which form a clique in the line graph since they all overlap in the vertices v_1 and v_2, but the line graph also has an isolated vertex corresponding to the red edge. This red edge is in p', but not in p'', and will be sent to a strict subpart of p by Υ.

Lemma 5.5. *For any representable endofunctor $\Phi_{\mathfrak{R}} : \mathcal{G} \to \mathcal{G}$ and any excisive clustering scheme $\mathfrak{C} : \mathcal{G} \to \mathcal{P}$, $\Phi_{\mathfrak{R}} \circ \mathfrak{C}$ is also an excisive clustering scheme.*

Proof. The proof of this result hinges on the following claim:

Claim 5.6. *For any $G = (V, E)$ and any $p \subseteq V$, $\Phi_{\mathfrak{R}}(G|_p) = \Phi_{\mathfrak{R}}(G)|_p$.*

Proof. Let $G = (V, E)$ be some graph, and let $p \subseteq V$ be arbitrary. We need to show that $\Phi_{\mathfrak{R}}(G|_p) = \Phi_{\mathfrak{R}}(G)|_p$. That they have the same vertex sets is obvious, and since $G|_p$ is a subgraph of G and $\Phi_{\mathfrak{R}}$ is a functor, we must have that $\Phi_{\mathfrak{R}}(G|_p)$ is a subgraph of $\Phi_{\mathfrak{R}}(G)$. So if we can show that any edge of $\Phi_{\mathfrak{R}}(G)|p$ is also an edge of $\Phi_{\mathfrak{R}}(G|_p)$, we will be done.

So suppose e is such an edge, and take $R \in \mathfrak{R}$ and $\omega : R \to G$ to witness this edge. We must have that $\omega(R) \subseteq p$ – but this means we can simply consider ω as a morphism into $G|_p$, and so it will also be an edge of $\Phi_{\mathfrak{R}}(G|_p)$, as needed. □

With this claim in hand, proving our lemma is nearly trivial – so let $G = (V, E)$ be some graph, and p be some part of $\mathfrak{C}(\Phi_R(G))$. We need to show that p is a part of $\mathfrak{C}(\Phi_R(G|_p))$. So if we let $H = \Phi_R(G)$, and observe that then $\Phi_R(G|_p) = H|_p$ by our claim and p is a part of $\mathfrak{C}(H)$ by assumption, the result is immediate by excisiveness of $\mathfrak{C}$. □

Proving that the k-ly connected components functor is excisive will be considerably trickier, because of the involvement of the non-functor Υ.

Proposition 5.7. *The k-ly connected components functor Π_k is excisive.*

Proof. Let $G = (V, E)$ be some graph, and assume $\Pi_k(G) = (V, P)$. Let $p \in P$ be some part – we need to show that p is a part of $\Pi_k(G|_p)$.

Let p' be the edge set of $G|_p$, and let p'' be the connected component of $\Lambda_k(G)$ which is sent to p by υ. Notice that we have $p'' \subseteq p'$, but it can be a strict subset when $k > 1$, as demonstrated by the example in Figure 5.

They key to this result is the following claim:

Claim 5.8. *For any set $q'' \subseteq E$ of edges, let q be the union of their vertex sets. Then it holds that $\Lambda_k(G)|_{q''}$ is a subgraph of $\Lambda_k(G|_q)$.*

Before we prove the claim, let us show why it will prove our proposition. By construction, the union of the vertex sets of the edges in p'' are precisely p, so our claim gives us that $\Lambda_k(G)|_{p''}$ is a subgraph of $\Lambda_k(G|_p)$. This in turn means, by functoriality of Π, that $\Pi(\Lambda_k(G)|_{p''})$ is a sub-partitioned-set of $\Pi(\Lambda_k(G|_p))$.

Now it is easy to see that $\Upsilon(\Pi(\Lambda_k(G)|_{p''}))$ and $\Upsilon(\Pi(\Lambda_k(G|_p)))$ have the same underlying set, and since we have just seen that the parts of $\Pi(\Lambda_k(G)|_{p''})$ are a subset of the parts of $\Pi(\Lambda_k(G|_p))$, we get that $\Upsilon(\Pi(\Lambda_k(G)|_{p''}))$ is a sub-partitioned-set of $\Upsilon(\Pi(\Lambda_k(G|_p)))$, and the latter is just $\Pi_k(G|_p)$.

So consider $\Upsilon(\Pi(\Lambda_k(G)|_{p''}))$. By assumption p'' is a connected component of $\Lambda_k(G)$, which is sent to p by υ, and so we must have that $\Upsilon(\Pi(\Lambda_k(G)|_{p''}))$ is precisely $(p, \{p\})$. So this shows that p is a part of $\Pi_k(G|_p)$, as desired.

So, let us proceed to give a proof of the claim.

Proof of Claim 5.8: Let q' be the edge set of $G|_q$. That $q'' \subseteq q'$ is obvious, so the vertices of $\Lambda_k(G)|_{q''}$ are a subset of the vertices of $\Lambda_k(G|_q)$. So suppose $e, f \in q''$ are adjacent in $\Lambda_k(G)$. By definition of the line graph functor, this means that $\mathfrak{v}(e) \cap \mathfrak{v}(f)$ has at least k members – but clearly $\mathfrak{v}(e), \mathfrak{v}(f) \subseteq q$, and so this intersection has the required size also in $G|_q$, and so these edges are adjacent also in $\Lambda_k(G|_q)$. □

With Lemma 5.5 and Lemma 5.7 in hand, it is an easy corollary that all representable clustering schemes are excisive.

Lemma 5.9. *Any representable clustering scheme is excisive.*

Lemma 5.10. *For any graph G, the parts of $\Pi_\infty(G)$ are precisely the vertex sets of the edges of G.*

Proof. This is trivial once we notice that $\Lambda_\infty(G)$ has no edges, so its connected components are all singleton sets consisting of a single vertex, that is, of a single edge of G. When we then apply υ to these, we get the vertex sets of these edges. □

Lemma 5.11. *Any excisive and functorial clustering scheme is refined by a representable clustering scheme.*

Proof. Let $\mathfrak{C}$ be some excisive and functorial clustering scheme, and let

$$\mathfrak{R} = \{G = (V, E) \in \mathcal{G} \mid V \text{ is a part of } \mathfrak{C}(G)\}.$$

We will show that $\Pi_{\mathfrak{R},\infty}$ refines $\mathfrak{C}$. So, let G be some graph. There are two things we need to show:

1. Every part p of $\mathfrak{C}(G)$ is also a part of $\Pi_{\mathfrak{R},\infty}(G)$.
2. Every part p of $\Pi_{\mathfrak{R},\infty}$ is contained in some part p' of $\Pi_{\mathfrak{R},\infty}(G)$.

First, suppose p is a part of $\mathfrak{C}(G)$. By excisiveness of $\mathfrak{C}$, $G|_p$ has p as a part and is thus in $\mathfrak{R}$. Now consider the inclusion morphism from $G|_p$ into G. This is a morphism from something in $\mathfrak{R}$ into G, and so there must be an edge of $\Phi_{\mathfrak{R}}(G)$ whose edge set is precisely p. It now follows from Lemma 5.10 that p is a part of $\Pi_\infty(\Phi_{\mathfrak{R}}(G)) = \Pi_{\mathfrak{R},\infty}(G)$.

In the other direction, suppose p is a part of $\Pi_{\mathfrak{R},\infty}(G)$. By Lemma 5.10, this must simply mean that there is an edge of $\Phi_{\mathfrak{R}}(G)$ whose vertex set is p. Now, this edge arises from a morphism ω from some $R \in \mathfrak{R}$ into G. By functoriality of $\mathfrak{C}$, this morphism will also be a morphism from $\mathfrak{C}(R)$ into $\mathfrak{C}(G)$ whose image is p. So by definition of what it means to be a morphism, there has to be a part of $\mathfrak{C}(G)$ containing p. □

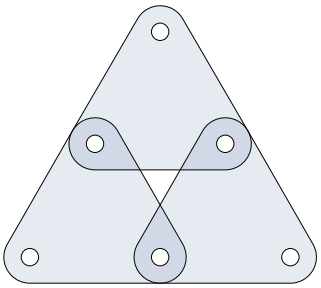


Figure 6: The graph D used in the proof of Lemma 5.16.

Finally, in order to show that there is no other implication that holds between these concepts, it suffices to give examples of clustering schemes with every combination of properties not already ruled out.

Example 5.12. The connected components functor Π_1 is representable and excisive – we can take its representation to be $\mathfrak{R} = \{E_n \mid n \in \mathbb{N}\}$, where E_n is the graph with n vertices and a single edge containing all vertices.

Example 5.13. For a scheme that is excisive but not functorial (and thus not representable), consider the scheme that partitions all graphs into a single part except K_2, which it partitions into two parts.

Example 5.14. For a scheme which is functorial but neither excisive nor representable, consider the scheme $\mathfrak{C}$ that is defined as follows:

1. If G contains any edge containing more than two vertices, cluster all of G as a single part.
2. Otherwise, if G is simple, for each connected component p of G:
   (a) If $|p| = 1$, cluster p as a single part.
   (b) If $|p| = 2$, and there is only one part of size two, cluster p as two parts. If there are multiple parts of size two, cluster each as a single part.
   (c) If $|p| > 2$, cluster p as a single part.

Example 5.15. Finally, for a scheme which has none of the properties, cluster all graphs as a single part except for K_2, which we cluster as two parts, and the disjoint union of two copies of K_2, which we cluster as two parts – that is, each of the copies is its own part. To see that this fails to be functorial, and thus also fails to be representable, consider the graph with two vertices and no edges, which is sent to a single part – but adding an edge between them splits the graph into two parts. To see that it fails to be excisive, consider the graph that is the disjoint union of two copies of K_2 – it has two parts, each of which is a copy of K_2, but these parts will in turn be clustered as two more parts, failing excisiveness.

Lemma 5.16. *There exists a clustering scheme which is excisive and functorial, but not equal to any representable clustering scheme.*

Proof. To define this clustering scheme, we begin by letting $\mathfrak{R}$ be the set containing only the graph D as in Figure 6. Now, for any G, consider the graph $\Phi_{\mathfrak{R}}(G)$. Each edge of this graph corresponds to a copy of D in G, and so for each vertex of $\Lambda_3(\Phi_{\mathfrak{R}}(G))$ we can give it three labels, namely the three edges of G that make up the corresponding copy of D. So we may define a graph $\Sigma(G)$ by taking $\Lambda_3(\Phi_{\mathfrak{R}}(G))$ and keeping an edge ef if and only if e and f share a label.

Concretely, this means that we will have an edge ef only if the three vertices they overlap in make up an edge of both copies of D to which they correspond.

Finally, we can define our clustering scheme $\mathfrak{C}$ by that $\mathfrak{C} = \Upsilon \circ \Pi \circ \Sigma$, analogously to how we normally define $\Pi_{\mathfrak{R},k}$, except here Σ plays the role of $\Lambda_3 \circ \Phi_{\mathfrak{R}}$.

So there are three things we will need to show:

1. $\mathfrak{C}$ is excisive,
2. $\mathfrak{C}$ is functorial, and
3. $\mathfrak{C}$ is not equal to any representable clustering scheme.

For the first two, the proofs are relatively straightforward adaptations of the proof that a representable clustering scheme is excsive and of functoriality of the line graph functor, respectively, and so we will only give a sketch of the proofs here. For the final point, we end up with a complicated mess of casework, that we defer to the appendix as Lemma B.1.

Excisiveness: Let G be some graph, and p be a part of $\mathfrak{C}(G)$. Recall that, from Claim 5.6, we have that $\Phi_{\mathfrak{R}}(G|_p) = \Phi_{\mathfrak{R}}(G)|_p$. So let p'' be the set of edges of $\Phi_{\mathfrak{R}}(G)$ that induce the part p. Then, entirely analogously to Claim 5.8, $\Sigma(G)|_{p''}$ is a subgraph of $\Sigma(G|_p)$. Clearly, $\Sigma(G)|_{p''}$ is a connected graph, and so $\Sigma(G|_p)$ must have a connected component containing p'' (in fact, p'' must be one of the connected components of $\Sigma(G|_p)$). This connected component is then sent to p by υ and so p is a part of $\mathfrak{C}(G|_p)$.

Functoriality: That Σ is a functor can be proven similarly to how we prove that Λ_k is a functor, only keeping track of a little bit more information about the labels of the edges – here it helps that we know $\Phi_{\mathfrak{R}}$ is a functor. Once we know that Σ is a functor, functoriality of $\mathfrak{C}$ follows by essentially the same proof as functoriality of Π_k. □

Remark 5.17. One might ask oneself why we have chosen to require all morphisms to be injective throughout. This is because in the non-overlapping case, one can relatively straightforwardly carry out the same arguments in the non-injective case, and find that there are then in fact only three different representable clustering schemes:

1. The scheme that always sends every vertex to its own part, represented by the empty set or by K_1,
2. the connected components functor, represented by any set of connected graphs containing at least one graph on at least two vertices,
3. and the scheme that always sends all vertices to the same part, represented by any set of graphs containing a disconnected graph.

Something similar seems likely to be true in the overlapping setting, but we choose not to pursue this here, to keep the structure of the paper simpler.

6 Computational complexity of representable clustering schemes

As stated in the introduction, one benefit of this approach to clustering is that it yields tractable algorithms for exactly computing the partitioning. In this section, we give a proof of this.

It should be clear that the only step in computing $\Pi_{\mathfrak{R},k}(G)$ that might be computationally hard is the step of computing $\Phi_{\mathfrak{R}}(G)$, since the other steps are all obviously computable in linear time.

The most naïve possible algorithm for computing $\Phi_{\mathfrak{R}}(G)$ given a fixed set $\mathfrak{R}$ of representing graphs and an n-vertex graph G is to, for each $\omega \in \mathfrak{R}$ and each $H \in \binom{V(G)}{|\omega|}$, that is, each $|\omega|$-sized subset H of the vertices of G, check whether $G|_H$ contains a subgraph isomorphic to ω. It is easy to see that this will give an algorithm that is polynomial in n, but with an exponent and constant depending on $\mathfrak{R}$.

What is more interesting is that we can get a quadratic time algorithm on any class of graphs of bounded expansion. Let us first recall what this means, before stating the lemma we will use.

Definition 6.1. A class of graphs $\mathcal{C}$ is said to have *bounded expansion* if for every $t \in \mathbb{N}$ there exists a c_t such that, whenever H is a shallow minor of depth t of some graph in $\mathcal{C}$, it holds that

$$\frac{|E(H)|}{|V(H)|} \leq c(t).$$

This notion generalizes both proper minor closed classes and classes of bounded degree, and can equivalently be defined in terms of low tree-depth decompositions.[NOW12]

To get our quadratic time algorithm, we will use the following result from [ND12, Corollary 18.2, p. 407]:

Lemma 6.2. *Let $\mathcal{C}$ be a class with bounded expansion and let H be a fixed graph. Then there exists a linear time algorithm which computes, from a pair (G, S) formed by a graph $G \in \mathcal{C}$ and a subset S of vertices of G, the number of isomorphs of H in G that include some vertex in S. There also exists an algorithm running in time $O(n) + O(k)$ listing all such isomorphs, where k denotes the number of isomorphs.*

Just applying this lemma for each $\omega \in \mathfrak{R}$ with $S = V(G)$ immediately gets us that we can, for fixed $\mathfrak{R}$, compute $\Phi_{\mathfrak{R}}(G)$ in polynomial time.

Theorem 6.3. *For any finite set $\mathfrak{R}$ of simple graphs and any class $\mathcal{C}$ of bounded expansion, there exists an algorithm that computes $\Phi_{\mathfrak{R}}(G)$ for any graph $G \in \mathcal{C}$, which runs in time $O\left(|V(G)| + |E(\Phi_{\mathfrak{R}}(G))|\right)$.*

Since we always have $|E(\Phi_{\mathfrak{R}}(G))| = O\left(n^{\rho}\right)$, where $\rho = \max_{\omega \in \mathfrak{R}} |V(\omega)|$, this algorithm is polynomial in n.

Remark 6.4. The bound on the number of edges in $\Phi_{\mathfrak{R}}(G)$ should be possible to improve in any class of bounded expansion, since we got it by just counting the number of subsets of $V(G)$ of size $|\omega|$ for each $\omega \in \mathfrak{R}$. Of course, for a sparse graph, it will not in fact be the case that every subset of $V(G)$ of size $|\omega|$ will contain a subgraph isomorphic to ω, so the bound drastically overestimates the number of edges there actually can be.

Maybe pursue this a little bit?

Acknowledgments

We thank Professors Tatyana Turova and Svante Janson and an anonymous reviewer for their helpful comments on a previous version of this paper.

References

[BAB12] Ahmad Barirani, Bruno Agard, and Catherine Beaudry. "Discovering and assessing fields of expertise in nanomedicine: a patent co-citation network perspective". In: *Scientometrics* 94.3 (Nov. 2012), pp. 1111–1136. DOI: 10.1007/s11192-012-0891-6. URL: https://doi.org/10.1007/s11192-012-0891-6 (cit. on p. 1).

[Bet23] Richard F. Betzel. "Chapter 7 - Community detection in network neuroscience". In: *Connectome Analysis*. Ed. by Markus D Schirmer, Tomoki Arichi, and Ai Wern Chung. Academic Press, 2023, pp. 149–171. ISBN: 978-0-323-85280-7. DOI: https://doi.org/10.1016/B978-0-323-85280-7.00016-6. URL: https://www.sciencedirect.com/science/article/pii/B9780323852807000166 (cit. on p. 1).

[CM13a] Gunnar Carlsson and Facundo Mémoli. "Classifying clustering schemes". In: *Foundations of Computational Mathematics* 13.2 (2013), pp. 221–252. DOI: 10.1007/s10208-012-9141-9 (cit. on p. 2).

[CM13b] Gunnar Carlsson and Facundo Mémoli. "Classifying clustering schemes". In: *Foundations of Computational Mathematics* 13.2 (Jan. 2013), pp. 221–252. DOI: 10.1007/s10208-012-9141-9. URL: https://doi.org/10.1007/s10208-012-9141-9 (cit. on p. 2).

[Cos+16] José M. Costa et al. "Sampling completeness in seed dispersal networks: When enough is enough". In: *Basic and Applied Ecology* 17.2 (2016), pp. 155–164. ISSN: 1439-1791. DOI: https://doi.org/10.1016/j.baae.2015.09.008. URL: https://www.sciencedirect.com/science/article/pii/S1439179115001255 (cit. on p. 1).

[CR10] Pu Chen and Sidney Redner. "Community structure of the physical review citation network". In: *Journal of Informetrics* 4.3 (2010), pp. 278–290. ISSN: 1751-1577. DOI: https://doi.org/10.1016/j.joi.2010.01.001. URL: https://www.sciencedirect.com/science/article/pii/S1751157710000027 (cit. on p. 1).

[Evk+21] Bojan Evkoski et al. "Evolution of topics and hate speech in retweet network communities". In: *Applied Network Science* 6.1 (Dec. 2021). DOI: 10.1007/s41109-021-00439-7. URL: https://doi.org/10.1007/s41109-021-00439-7 (cit. on p. 1).

[FB07] Santo Fortunato and Marc Barthélemy. "Resolution limit in community detection". In: *Proceedings of the National Academy of Sciences of the United States of America* 104.1 (Jan. 2007), pp. 36–41. DOI: 10.1073/pnas.0605965104. URL: https://doi.org/10.1073/pnas.0605965104 (cit. on p. 2).

[Jav+18] Muhammad Aqib Javed et al. "Community detection in networks: A multidisciplinary review". In: *Journal of Network and Computer Applications* 108 (2018), pp. 87–111. ISSN: 1084-8045. DOI: https://doi.org/10.1016/j.jnca.2018.02.011. URL: https://www.sciencedirect.com/science/article/pii/S1084804518300560 (cit. on p. 1).

[JS71] N. Jardine and R. Sibson. *Mathematical Taxonomy*. Wiley series in probability and mathematical statistics. Wiley, 1971. ISBN: 9780471440505. URL: https://books.google.se/books?id=ka4KAQAAIAAJ (cit. on p. 1).

[Koj+23] Sadamori Kojaku et al. *Network community detection via neural embeddings*. 2023. arXiv: 2306.13400 [physics.soc-ph] (cit. on p. 1).

[ND12] Jaroslav Nešetřil and Patrice Ossona De Mendez. *Sparsity*. Jan. 2012. DOI: 10.1007/978-3-642-27875-4. URL: https://doi.org/10.1007/978-3-642-27875-4 (cit. on p. 19).

[NG04] Michelle G. Newman and Michelle Girvan. "Finding and evaluating community structure in networks". In: *Physical Review E* 69.2 (Feb. 2004). DOI: 10.1103/physreve.69.026113. URL: https://doi.org/10.1103/physreve.69.026113 (cit. on p. 1).

[NOW12] Jaroslav Nešetřil, Patrice Ossona de Mendez, and David R. Wood. "Characterisations and examples of graph classes with bounded expansion". In: *European Journal of Combinatorics* 33.3 (2012). Topological and Geometric Graph Theory, pp. 350–373. ISSN: 0195-6698. DOI: https://doi.org/10.1016/j.ejc.2011.09.008. URL: https://www.sciencedirect.com/science/article/pii/S0195669811001569 (cit. on p. 19).

[Pei14] Tiago P. Peixoto. "Hierarchical block structures and High-Resolution model selection in large networks". In: *Physical Review X* 4.1 (Mar. 2014). DOI: 10.1103/physrevx.4.011047. URL: https://doi.org/10.1103/physrevx.4.011047 (cit. on p. 1).

[RAB09] Martin Rosvall, Daniel Axelsson, and Carl T. Bergstrom. "The map equation". In: *European Physical Journal-special Topics* 178.1 (Nov. 2009), pp. 13–23. DOI: 10.1140/epjst/e2010-01179-1. URL: https://doi.org/10.1140/epjst/e2010-01179-1 (cit. on p. 1).

A Definitions of categorical language

For the reader of a more combinatorial bent, who might not recall all the definitions of category theory used in this paper, we include a short appendix stating them.

Definition A.1. A *category* $\mathcal{C}$ consists of a class of *objects* $\mathrm{ob}(\mathcal{C})$, and for each pair of objects $a, b \in \mathcal{C}$ a (possibly empty) class $\mathrm{Hom}(a, b)$ of *morphisms* from a to b.

We require these morphisms to compose, in the sense that for any $f \in \mathrm{Hom}(a, b)$ and $g \in \mathrm{Hom}(b, c)$ there exists an $fg \in \mathrm{Hom}(a, c)$, and this composition must be associative. There must also, for each $a \in \mathcal{C}$, be an identity morphism $\mathrm{id}_a \in \mathrm{Hom}(a, a)$, with the property that $f\mathrm{id}_a = f$ and $\mathrm{id}_a g = g$ whenever these compositions make sense.

It is worth remarking that the use of the word *class* instead of *set* is not accidental – the collection of all finite simple graphs is of course too big to be a set. However, since this is

only so for "dumb reasons" – that there are class-many different labelings of the same graph – and the collection of isomorphism classes of finite simple graphs *is* a set, this distinction will never actually affect us, and we will elide it throughout the text.

Definition A.2. A *functor* F from a category $\mathcal{A}$ to a category $\mathcal{B}$ is a mapping that associates to each $a \in \mathcal{A}$ an $F(a) \in \mathcal{B}$, and that for each pair of objects $a, a' \in \mathcal{A}$ associates each morphism $f \in \mathrm{Hom}(a, a')$ to a morphism $F(f) \in \mathrm{Hom}(F(a), F(a'))$. We require this mapping to respect the identity morphisms, in the sense that $F(\mathrm{id}_a) = \mathrm{id}_{F(a)}$, and composition of morphisms, so that $F(fg) = F(f)F(g)$.

A functor from a category to itself is called an *endofunctor*.

The categories we study in this paper are all concrete, that is, their objects are sets with some extra structure on them, and the morphisms are just functions between the sets that respect the structure in an appropriate sense. Our functors never do anything strange – they all leave the underlying set unchanged, only changing what structure we have on the set, and they do "nothing" on the morphisms, that is, as set functions they are just the same function again between the same sets.

In this setting most of the requirements of a functor are trivial – of course it will behave correctly with identity morphisms and composition, because in a sense it isn't doing anything to the morphisms. Therefore, the one thing we need to check when proving that things are functors will be that morphisms do map to morphisms, that is, that a function that respects the structure on the sets in the first category will also respect the structure we have in the second category.

The final categorical concept we will need is that of two *functors* being isomorphic, since our classification result ends up giving us not equality but just isomorphism of clustering schemes.

Definition A.3. Suppose $\mathcal{A}$ and $\mathcal{B}$ are two categories, and F and G are two functors from $\mathcal{A}$ to $\mathcal{B}$. A *natural transformation* η from F to G is a mapping that associates to each object $a \in \mathcal{A}$ an element $\eta_a \in \mathrm{Hom}(F(a), G(a))$, such that for every object $b \in \mathcal{A}$ and each morphism $f \in \mathrm{Hom}(a, b)$, the following diagram commutes:

$$\begin{array}{ccc} F(a) & \xrightarrow{F(f)} & F(b) \\ {\scriptstyle \eta_a}\downarrow & & \downarrow{\scriptstyle \eta_b} \\ G(a) & \xrightarrow[G(f)]{} & G(b) \end{array}$$

In other words, the following equation must hold for all $f \in \mathrm{Hom}(a, b)$:

$$\eta_b \circ F(f) = G(f) \circ \eta_a.$$

Two functors F and G are *isomorphc* if there exists a natural transformation η from F to G such that η_a is an isomorphism for all $a \in \mathcal{A}$.

This notion is very general – in our actual result, both $\mathcal{A}$ and $\mathcal{B}$ will be concrete categories, and for each a, a, $F(a)$ and $G(a)$ will have the same underlying set, and f, $F(f)$, and $G(f)$ will be the same as functions on the underlying sets. So here we will be able to take each η_a to be the identity function on the underlying set. So all the notation of the definition ends up referring to very simple things, in the end.

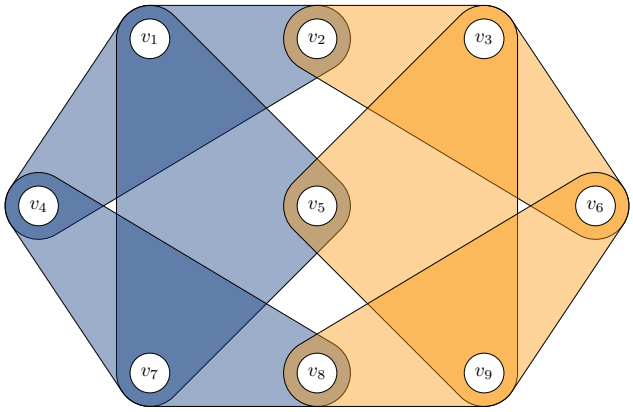


Figure 7: The graph G used in the proof of Lemma B.1.

B Proof of Lemma B.1

Lemma B.1. *The clustering scheme defined in Lemma 5.16 is not equal to any representable clustering scheme.*

Proof. Suppose for contradiction that $\mathfrak{C} = \Pi_{\mathfrak{R},k}$ for some $\mathfrak{R}$ and k, and assume we take $\mathfrak{R}$ of minimal cardinality. We need to exclude every possible combination of $\mathfrak{R}$ and value of k – so we end up with a lot of casework.

$\mathfrak{R}$ cannot contain any proper subgraph of D, and must contain D itself: It is clear that $\mathfrak{R}$ cannot contain any graph G which does not have D as a subgraph, since then $\Pi_{\mathfrak{R},k}(G)$ would have a part containing all vertices, while $\mathfrak{C}(G)$ would have no parts at all.

In particular this means $\mathfrak{R}$ cannot contain any proper subgraph of D, and so, in order for $\Pi_{\mathfrak{R},k}(D)$ to have a part containing all vertices, we must have $D \in \mathfrak{R}$.

Excluding the case $k \leq 3$: In this case, consider the graph G as in Figure 7. An attentive eye will see that this graph is two copies of D, one on the blue edges and one on the orange edges, and they overlap in their corners. It is clear that $\mathfrak{C}(G)$ will have two parts, since the overlap of the two copies of D is not an edge of either copy. However, since $D \in \mathfrak{R}$ and $k \leq 3$, we must have a part containing all vertices in $\Pi_{\mathfrak{R},k}(G)$, which is a contradiction.

Establishing graphs that have to be in $\mathfrak{R}$ if k is big: Now consider the family F_i of graphs as in Figure 8. We claim that if $k > 3i$, we must have $F_i \in \mathfrak{R}$. We proceed by induction – so, for the base case, suppose that $k > 3$ and consider the graph F_1. It is easy to see that $\mathfrak{C}(F_1)$ has a single part containing all vertices. Now, $\Phi_{\mathfrak{R}}(F_1)$ contains at least two edges corresponding to the two copies of D in F_1, but these overlap in only three vertices, and so there will not be an edge between them in $\Lambda_k(\Phi_{\mathfrak{R}}(F_1))$. So there must be some additional graph G in $\mathfrak{R}$ that is a subgraph of F_1, and overlaps both copies of D in more than three vertices.

If this G is a proper subgraph of F_1, it is easy to see that it will not be sent to a single part by $\mathfrak{C}$, which would be a contradiction to the fact that $\Pi_{\mathfrak{R},k}(G)$ would send this G to a single part. So G must be F_1 itself, and so $\mathfrak{R}$ must contain F_1.

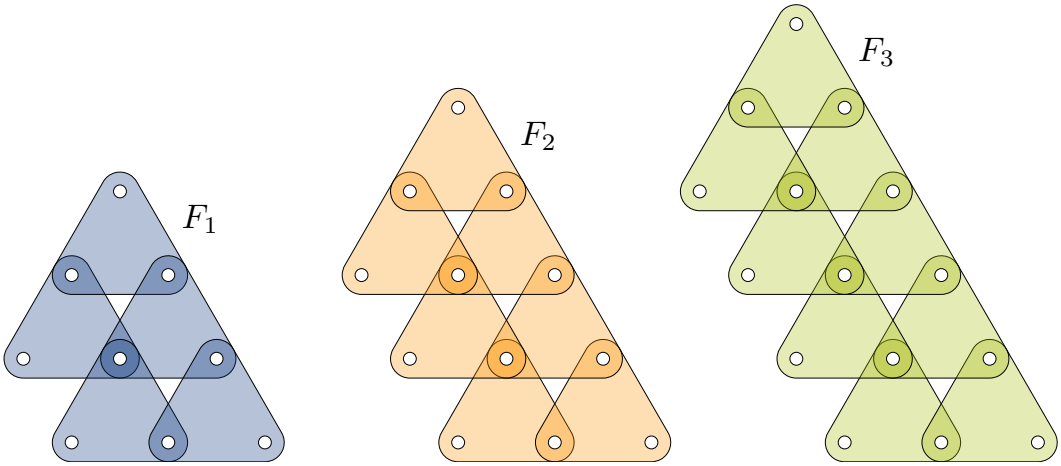


Figure 8: The graph family F_i used in the proof of Lemma B.1, illustrated by its first three members.

For the inductive step, let us fix an i and suppose that $k > 3i$. Then, by our induction hypothesis, $F_{i-1} \in \mathfrak{R}$. Again, we have that $\mathfrak{C}(F_i)$ has a single part containing all vertices. In $\Phi_{\mathfrak{R}}(F_i)$ we will have two edges corresponding to the copy of F_{i-1} in F_i created by the first i copies of D and last i copies of D respectively, and these overlap in a copy of F_{i-2}, which has $3i < k$ vertices. So there is no edge connecting these two edges in $\Lambda_k(\Phi_{\mathfrak{R}}(F_i))$, and so there must be some additional graph G in $\mathfrak{R}$ that is a subgraph of F_i and overlaps both copies of D in more than $3i$ vertices. The same argument again gives that, if this subgraph were proper, it would be a copy of F_{i-1} with a single edge hanging off of it, which would not be sent to a single part by $\mathfrak{C}$, and so it must be F_i itself.

Excluding the case $3 < k < \infty$: So, suppose $3 < k < \infty$, and let i be such that $3i < k \leq 3(i+1)$. From the preceding section, we know that $F_i \in \mathfrak{R}$. Our strategy will be to find a graph G_i that has two copies of F_i in it which overlap in all but three vertices, but share no edge. Now, since F_i has $3(i+2)$ vertices, and so the two copies of F_i would be overlapping in $3(i+1) \geq k$ vertices, we would have that $\Pi_{\mathfrak{R},k}(G_i)$ would have a part containing all vertices, while $\mathfrak{C}(G_i)$ would not, since the two copies of F_i do not overlap in any full edge.

The construction we use here gets too complicated to draw in any case beyond $i = 1$ (that case is illustrated in Figure 9), but the idea is the following: The graph F_i has three "columns" of vertices, and we can take our two copies of F_i to have the same left- and rightmost columns. For the middle column, we pick three of the vertices in each copy to be distinct, and for the rest, we permute their labels and then identify vertices with the same label. As long as we pick a permutation σ such that $|\sigma(j) - j| > 1$ for all i, we will have that the two copies of F_i do not share any edges.

That such a permutation σ exists for $i > 3$ is easy to see – in that case, we can even pick such a permutation of the entire middle column first. In the case $i = 1$, there are no middle column vertices left once we remove the three distinct ones, and for the cases $i = 2, 3$, the reader is invited to look at Figure 8 and verify that the construction can be performed then as well.

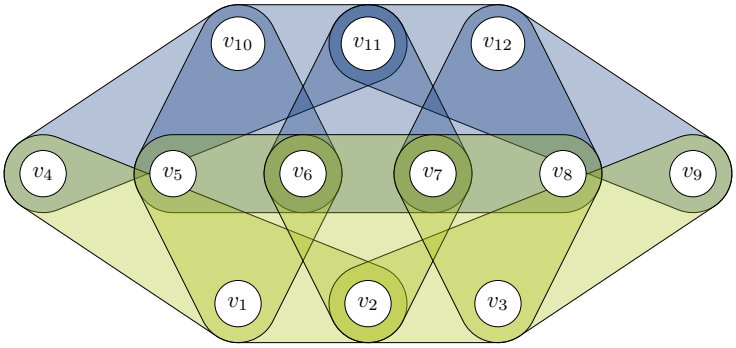


Figure 9: The graph G_1 used in the proof of Lemma B.1. This graph contains two copies of F_1, corresponding to the green and the blue edges. Here, the vertices v_4 and v_9 are the leftmost column in the standard drawing of F_1 (Figure 8), and the vertices v_5, v_6, v_7 and v_8 are the rightmost column. The vertices v_1, v_2, v_3 are the middle column in the green copy of F_1, and the vertices v_{10}, v_{11}, v_{12} are the middle column in the blue copy of F_1.

Excluding the case $k = \infty$: Finally, we need to exclude the case $k = \infty$. In this case, we know that $\mathfrak{R}$ must be precisely the set of all graphs that are sent to a partition with a part containing all vertices by $\mathfrak{C}$, from the reasoning in the proof of Lemma 5.11. But then, for example, the graph F_1 will be sent to a partition that has three parts – one containing all vertices, and two corresponding to the copies of D in F_1, while of course $\mathfrak{C}(F_1)$ has only one part. So we have a contradiction. □